

插入排序法、冒泡排序、选择排序三种排序方法各自的优越点，其时间和空间复杂度？

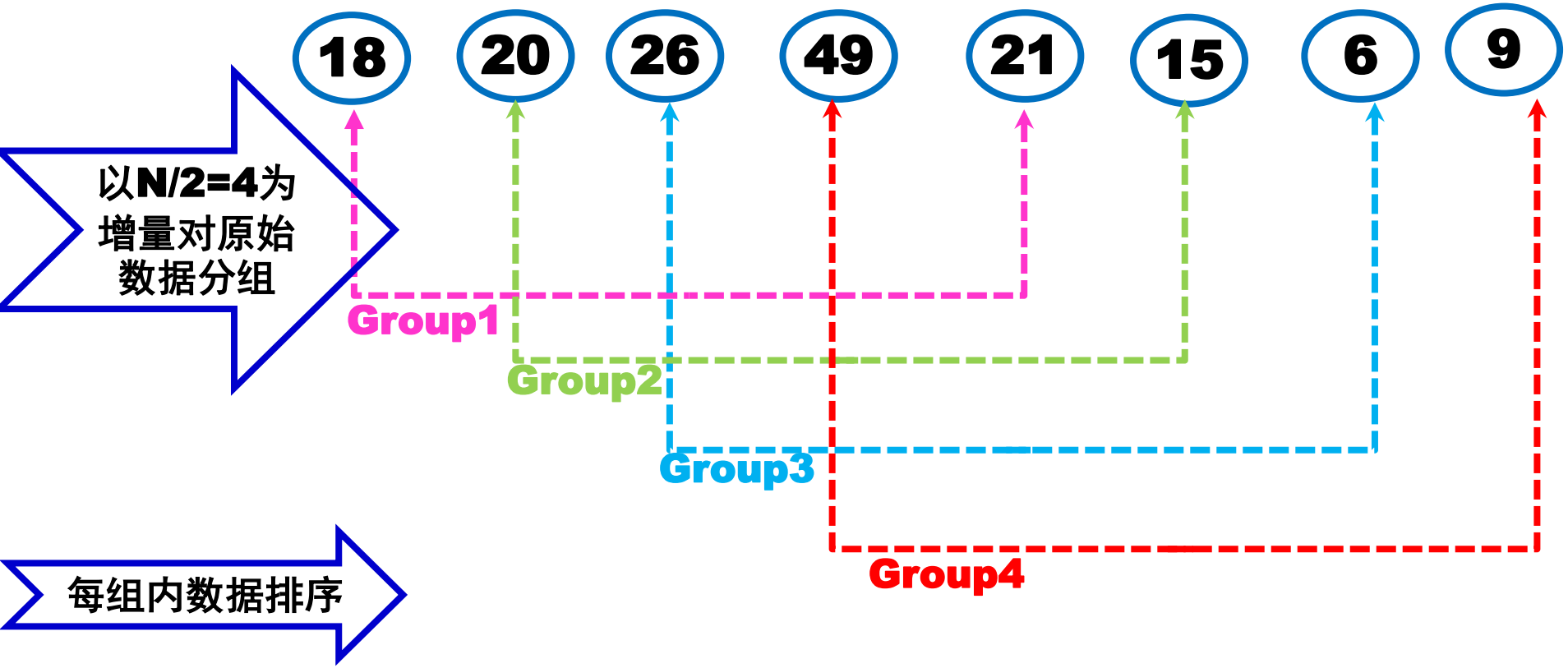
算法	平均时间复杂度	最好情况	最坏情况	空间复杂度	稳定性
插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(n)$	稳定
冒泡	$O(n^2)$	$O(n)$	$O(n^2)$	$O(n)$	稳定
选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)$	不稳定

常用的排序法（2）

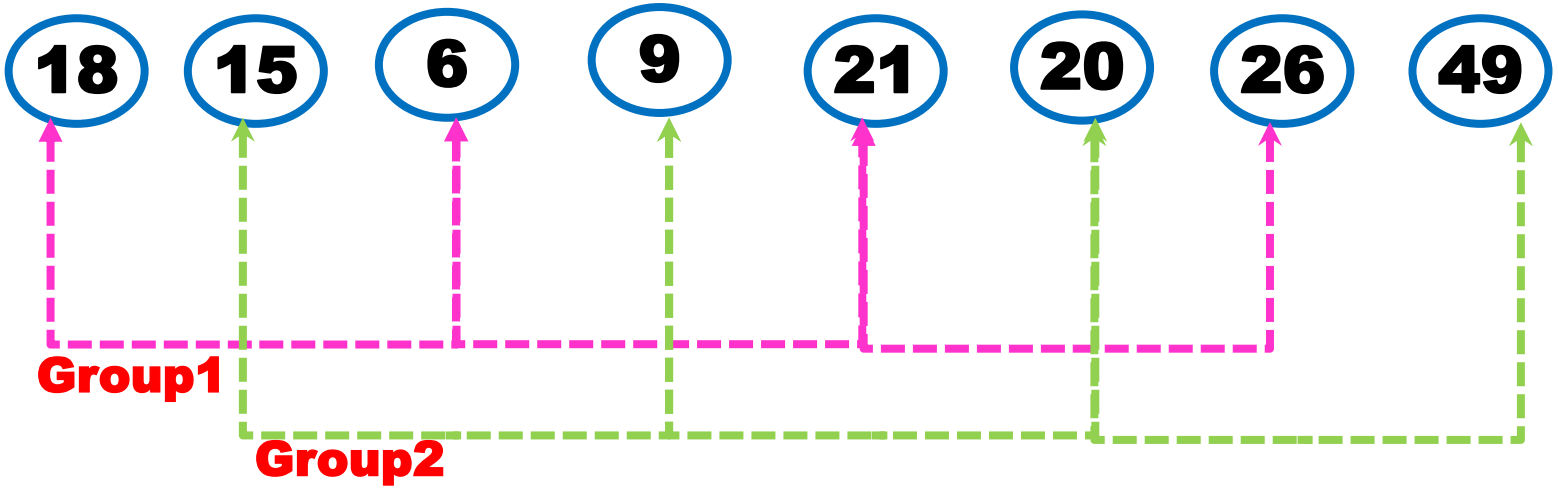
- 希尔排序
- 快速排序
- 归并排序

5.2、排序-希尔排序

缩小增量排序法



5.2、排序-希尔排序



以(N/2)/2为增量对原始数据分组

对每组内数据排序

5.2、排序-希尔排序



每一趟排序包含若干子序列，其中第一个子序列第一个元素是0号，第二个元素是gap号。

每趟排序从第一个子序列开始直接插入排序，但不是完成一个子序列，再做下一个子系列，而是先做每个子序列的第一步，再做每个子序列的第二步，。。，穿插完成。

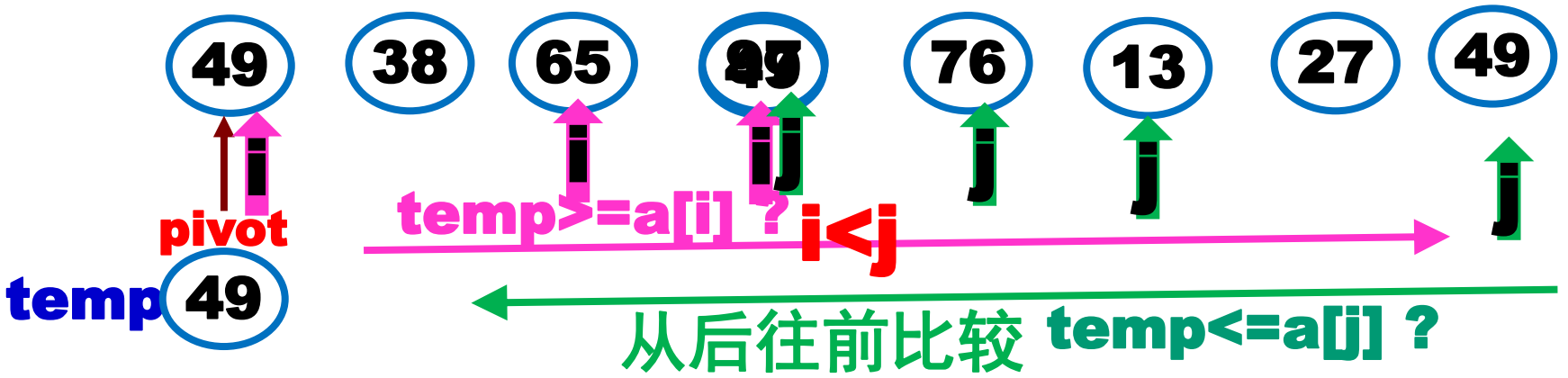
信息科学与工程学院 王文博 2026

5.2、排序-希尔排序算法实现

```
int gap=(last+1)/2;
while(gap)
{
        for( int i=gap;i<=last;i++)
        {
                T temp=slist[i];
                int j=i;
                while(j>=gap && temp<slist[j-gap])
                {
                        slist[j]=slist[j-gap];
                         j-=gap;
                }
                slist[j]=temp;
        }
    gap/=2;
}
```

5.2、排序-快速排序

从数组中选择一个元素，我们把这个元素称之为**基准元素**，然后把数组中所有小于基准元素的元素放在其左边，所有大于或等于基准元素的元素放在其右边



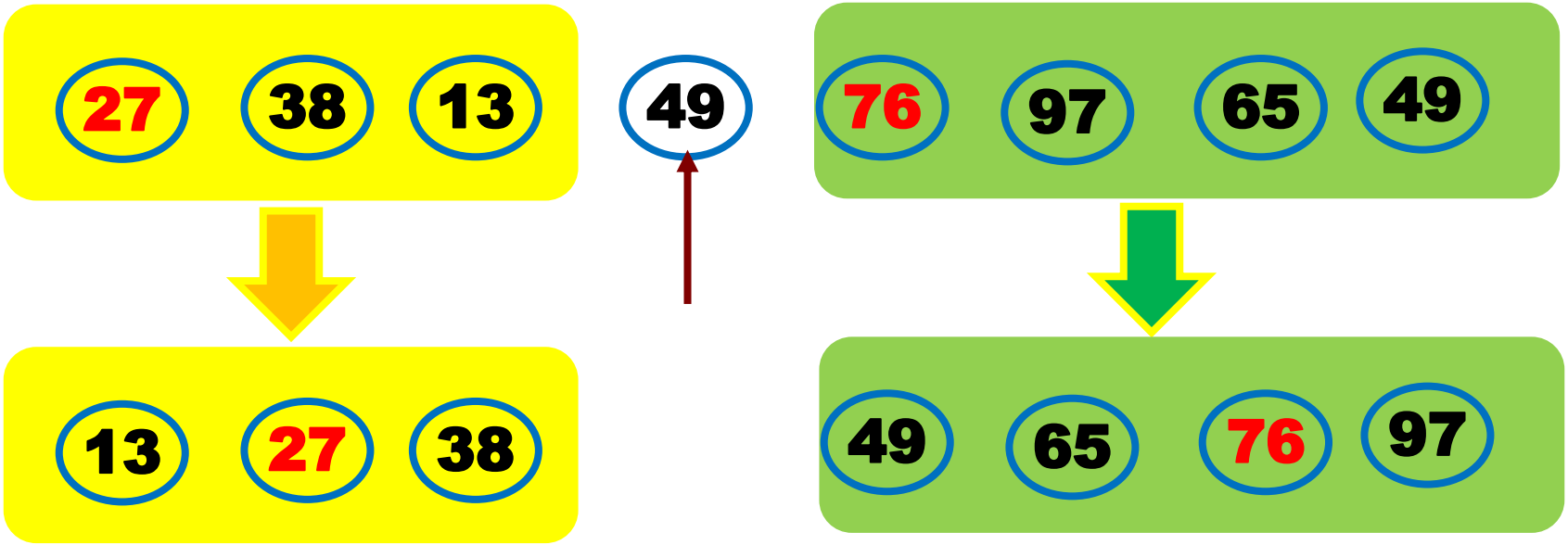
- 一趟排序后，基准元素在准确的排序位置上

5.2、排序-快速排序

```
i=low; j=high;
temp=slist[low];
while(i<j)
{
        while(temp<=slist[j]&& (i<j)) j--;
        if(i<j)
        {
            slist[i]=slist[j];
            i++;
        }
        while(temp>slist[i] && i<j) i++;
        if(i<j)
        {
                slist[j]=slist[i];
                j--;
        }
}
slist[i]=temp;
```

5.2、排序-快速排序

可以通过递归的方式，让基准元素左边的数组和右边的数组也重复同样的操作，直到数组的大小为1，此时每个元素都处于**其准确的位置上**。



5.2、排序-归并排序

基本思想：（分而治之的思想）

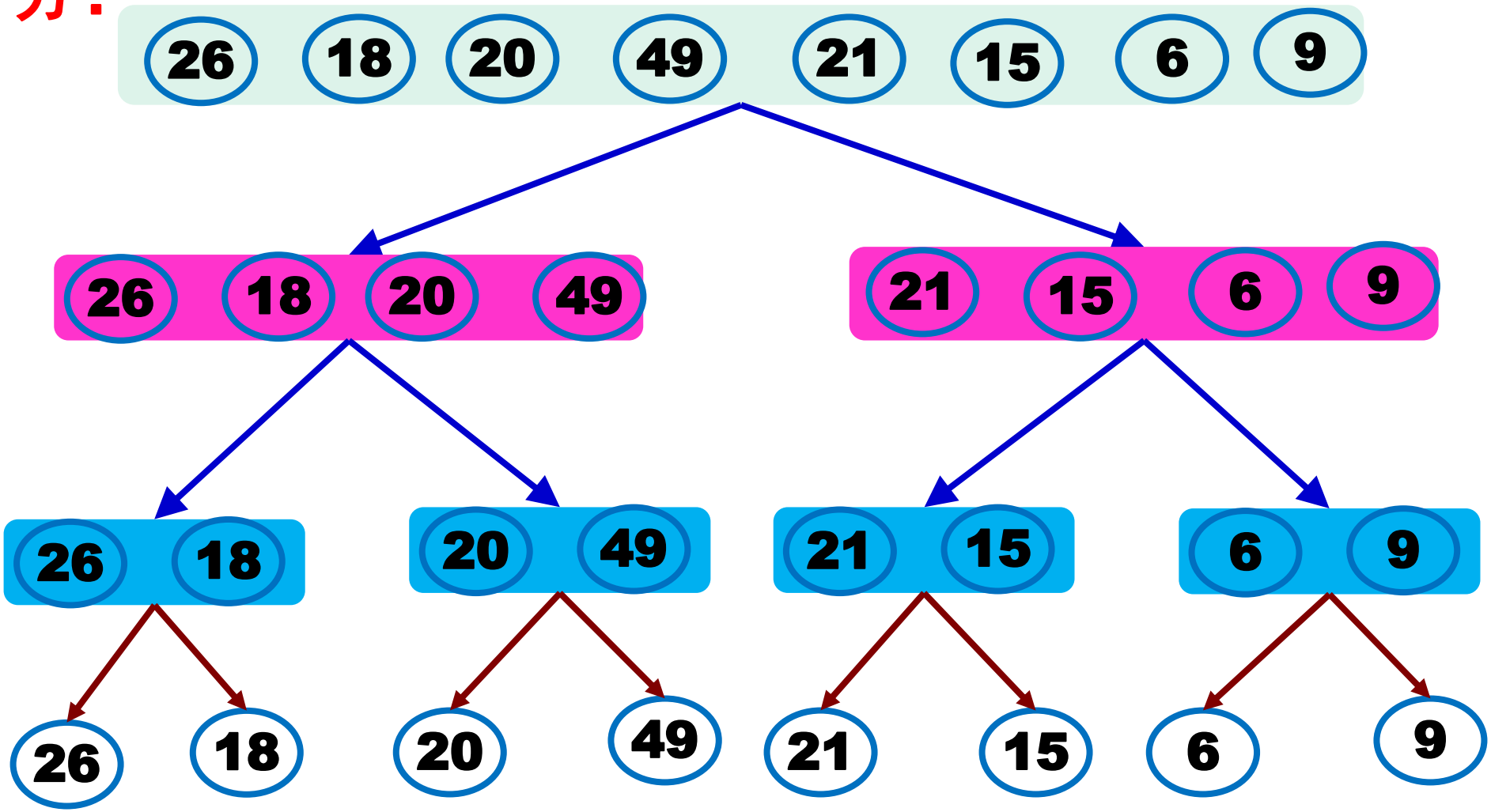
把大的数组分成两个，然后对这两个数组分别进行排序，之后在把这两个数组合并成一个有序的数组。

由于两个小的数组都是有序的，所以在合并的时候是很快的。

可通过**递归**的方式将大的数组一直分割，直到数组的大小为 1，只有一个元素，是有序的， 之后再把两个数组大小为1的合并成一个大小为2的，再把两个大小为2的合并成4的 …. . 直到全部小的数组合并起来。

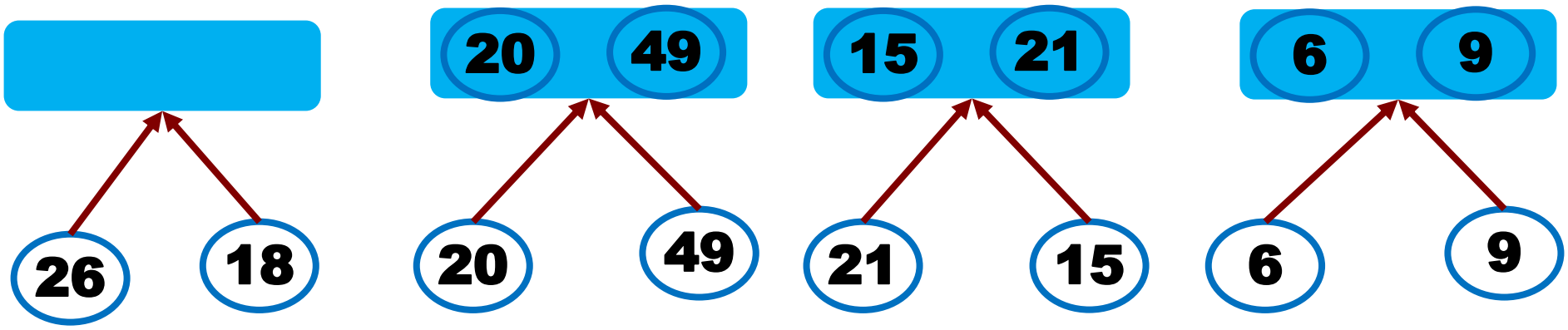
5.2、排序-归并排序

分：


26 18 20 49 21 15 6 9
26 18 20 49
21 15 6 9
26 18
20 49
21 15
6 9
26
18
20
49
21
15
6
9

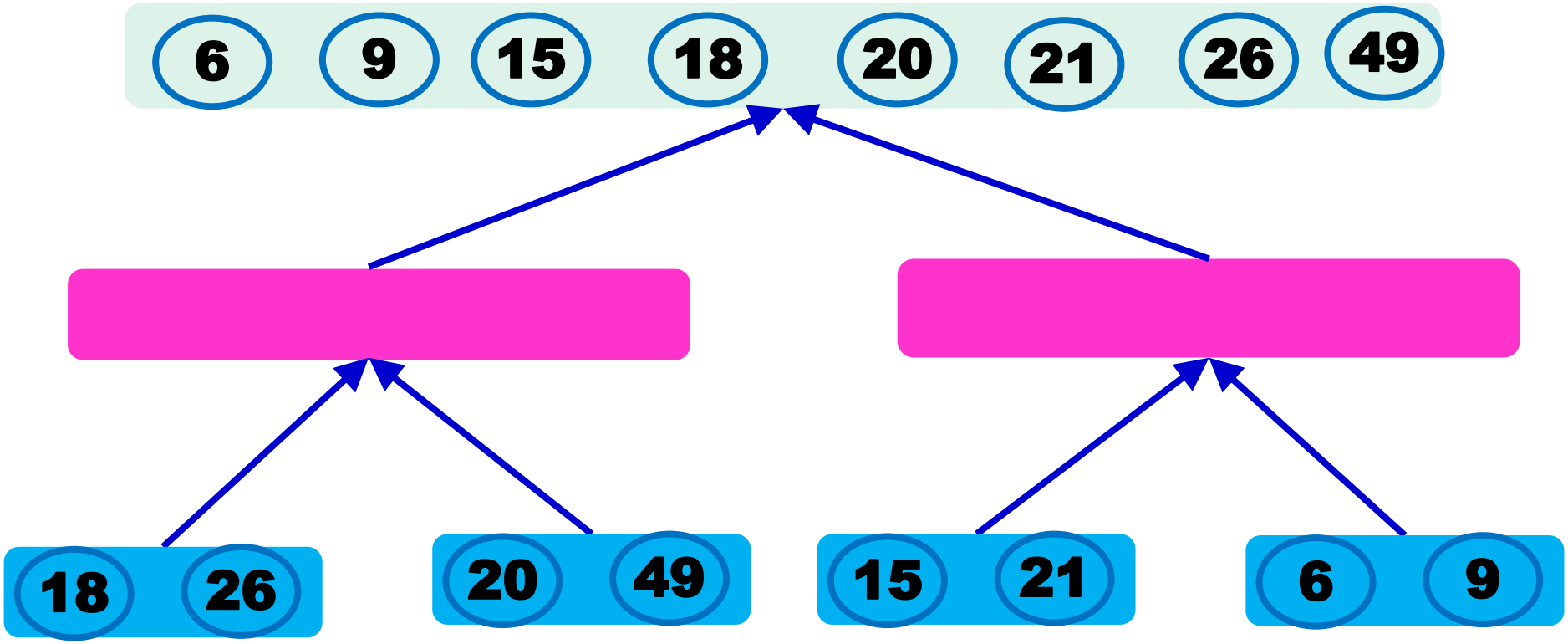
5.2、排序-归并排序

合：



5.2、排序-归并排序

合：



核心算法:两个有序的数组合并为一个有序数组

6、模板与类参数

函数模板常用方式：

- **(1)** 函数模板作为类模板的成员函数。
- **(2)** 独立的非成员函数。

要点：理解(类型参数) (参数化的类型)

冒泡排序算法 (作为成员函数)

```
template <typename T,int size> class Orderedlist
{
/*无关数据成员和成员函数省略*/
public:
    void BubbleSort();
    void print();
};
template <typename T,int size>
void Orderedlist<T,size>::BubbleSort()
{
   bool noswap;
   /*算法过程详见例6.8*/
} /*其他成员函数的定义略*/
```

冒泡排序算法 (作为成员函数)

```
int main()
{
    const int h=10;
    int i;
    Orderedlist<mystring,100> ordlist;
    mystring n[h] ={"cat","book","car","zoo","fish",
                    "cab", "dog","cap","fox","can"};
    for(i=0;i<h;i++)  ordlist.Insert (n[i], i);
    cout<<“未排序数组： "<<endl;
    ordlist.print();
    ordlist.BubbleSort();
    cout<<“已排序数组： "<<endl;
    ordlist.print();
    return 0;
}
```

【例】冒泡排序算法(独立的非成员函数)

```
template <typename T>
void BubbleSort(T* slist, int n)
{
    bool noswap;
    int i,j;
    T temp;

    //.....交换排序过程略

 }
/*独立的非成员函数*/
```

冒泡排序算法 (作为独立非成员函数)

字符串类定义如下：

```
class mystring
{
    char str[21];
    int maxsize;
    int last;
public:
    mystring() {last=-1;maxsize=21;str[0]='\0';    }
    mystring(char *s);
    ~mystring(){}
    void show(){cout<<str<<'\t';} //显示也必须在数据类定义
    bool operator<(mystring &); //重载<
    mystring & operator=(char * ms); //赋c字符串给mystring
};
……//成员函数定义略
```

冒泡排序算法(独立的非成员函数)

```
int main()
{
    const int h=10;
    int i;
    mystring list[h]={"cat","book","car","zoo",
                      "fish","cab", "dog",
                    "cap","fox","can"};
    cout<<“未排序数组： "<<endl;
    for(i=0;i<h,i++) list[i].show();
    BubbleSort(list,h);
    cout<<“已排序数组： "<<endl;
    for(i=0;i<h,i++) list[i].show();
    return 0;
}
```

梯形法求积分(成员函数)

```
template<typename T>class Integer{
        double a,b,step,result;
        int n;   T cf;
  public:
    Integer(double aa=0, double bb=0, int nn=100)
    {    a=aa;  b=bb;  n=nn;          integerate();   }
    void putlimits(double aa=0, double bb=0,
     int nn=100){      a=aa;    b=bb;          n=nn;  }
    void integerate();
    void print(){cout<<"定积分值为： "<<result<<endl;}
};
template<typename T>void Integer<T>::integerate(){
        step=(b-a)/n;
        result=(cf.fun(a)+cf.fun(b))/2;
        for (int i=1;i<n;i++) result+=cf.fun(a+i*step);
        result*=step;
}
```

梯形法求积分(成员函数)

```
int main(){
    Integer<F1> integer1(0.0,3.0,1000);
    integer1.print();
    Integer<F2> integer2(0.0,3.0,1000);
    integer2.print();
    Integer<F3> integer3(0.0,3.0,1000);
    integer3.print();
    return 0;
}
```

a=0.0;
b=3.0;
n=1000;
integerate()

```
class F1 {
public:
double fun(double x)
      {return (1+x+2*x*x);}
};
```

```
class F2 {
public:
double fun(double x){return(1+x+2*x*x+3*x*x*x);}
};
```

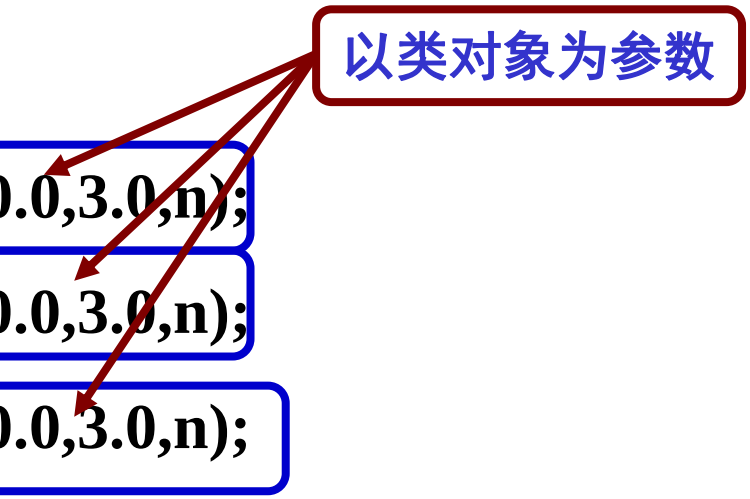
```
class F3 {
public:
double fun(double x){return (1+x+2*x*x +3*x*x*x)+4*x*x*x*x);}   };
```

梯形法求积分(独立的非成员函数)

```
template<typename T>double integer (T cf,float a, float b,int n){
        double result,step;
        result=(cf.fun(a)+cf.fun(b))/2;
        step=(b-a)/n;
        for (int i=1;i<n;i++) result+=cf.fun(a+i*step);
        result*=step;
        return result;
}
```

梯形法求积分(独立的非成员函数)

```
int main(){
        F1 f1;F2 f2;F3 f3;
        double fixint1, fixint2, fixint3;
        int n=1000;
        fixint1=integer(f1,0.0,3.0,n);
        fixint2=integer(f2,0.0,3.0,n);
        fixint3=integer(f3,0.0,3.0,n);
        cout<<fixint1<<'\n'<<fixint2<<'\n'<<fixint3<<'\n';
        return 0;
}
```



梯形法求积分(函数指针法)

```
double  f1(double x)  {return (1+x+2*x*x);}
double  f2(double x){return 1+x+2*x*x+3*x*x*x;}
double  f3(double x){return 1+x+2*x*x +3*x*x*x)+4*x*x*x*x;}
double integer (double (*fun)(double)，float，float，int);
 int main(){
        double fixint1, fixint2, fixint3;
        int n=1000;
        fixint1=integer(f1,0.0,3.0,n);
        fixint2=integer(f2,0.0,3.0,n);
        fixint3=integer(f3,0.0,3.0,n);
        cout<<fixint1<<'\n'<<fixint2<<'\n'<<fixint3<<'\n';
        return 0;
}
```

调用函数，实参为函数名f1

梯形法求积分(函数指针法)

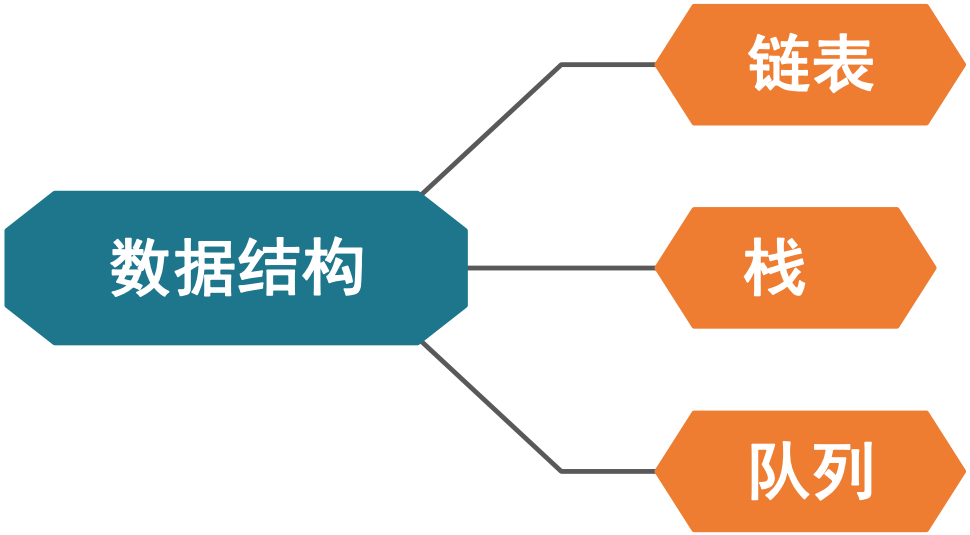
```
double integer (double (*fun)(double)，float，float，int)
{
        double result,step;
        result=fun(a)+fun(b))/2;
        step=(b-a)/n;
        for (int i=1;i<n;i++) result+=fun(a+i*step);
        result*=step;
        return result;
}
```

形参用指向函数的指针变量fun接受函数名

用fun调用函数，实际调用的是实参函数

第七章 动态内存分配与数据结构





1、自由存储区内存管理

自由存储区/堆区（动态数据）：

存放与指针相关数据，初始化时随机取值。

运行时确定

栈区(函数局部数据/main()函数局部数据)：

初始化时随机取值。

全局数据区（全局变量、静态变量）：

变量的所有字节自动初始化为0。

代码区（程序代码）：

存放程序代码。

编译时预订存储空间

1.1、自由存储区内存管理

C++可在程序运行过程中向自由存储区（堆区）动态申请和释放的一块存储空间

new和delete运算符

- 运行时遇到需要分配的空间的情况，申请：**new**
- 不再使用该空间时，需要显式释放该空间：**delete**

1.2、自由存储区内存管理-new运算符

new 运算符的一般格式为： 指针变量= new 类型 <(初值)>

- ✓ new 在堆区成功分配空间后，返回这个空间的首地址
- ✓ 这个地址须用指针保存

```
int *p1,*p2,*p3;
```

p1 p2 p3

```
p1= new int;
```

在堆区申请分配存放1个int型数据的存储空间，4个字节

new申请的空间

```
p2= new int(4);
```

在堆区申请分配存1个放int型的数据的存储空间，初值为4

4

new申请的空间

```
p3= new int[4];
```

可以是变量或表达式

在堆区申请分配存放4个int型元素的数组的内存空间

new申请的空间

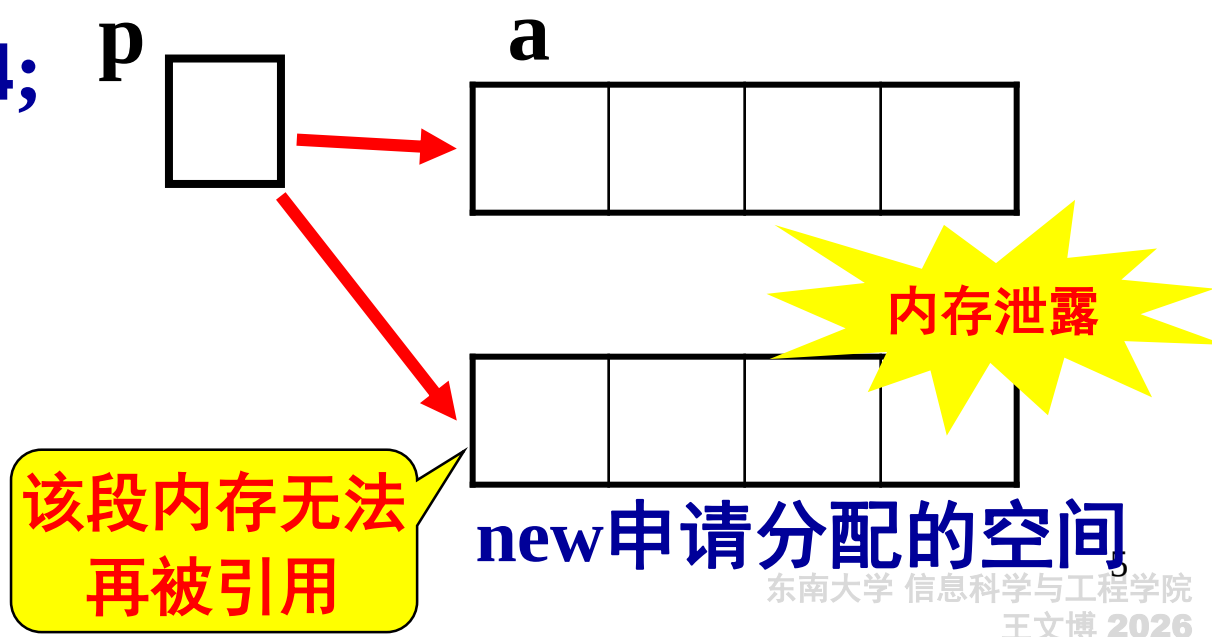
王文博 2026

1.2、自由存储区内存管理-new运算符

- 当内存中没有足够的空间给予分配时，new 运算符返回空指针NULL（0）

- 指向其首地址的指针是引用new开辟的内存单元唯一途径，若指针变量重新赋值，可能会有内存泄露

```
int  * p, a[4],n=4;
p=new int[n];
p=a;
```



东南大学 信息科学与工程学院
王文博 2026

1.3、自由存储区内存管理-delete运算符

用new开辟的内存单元如果程序不“主动”收回，那么这段空间就一直存在，会造成内存浪费和泄露，delete运算符用来将动态分配的内存空间归还给系统。

delete 运算符的一般格式为：

delete <[ ]> 指针变量

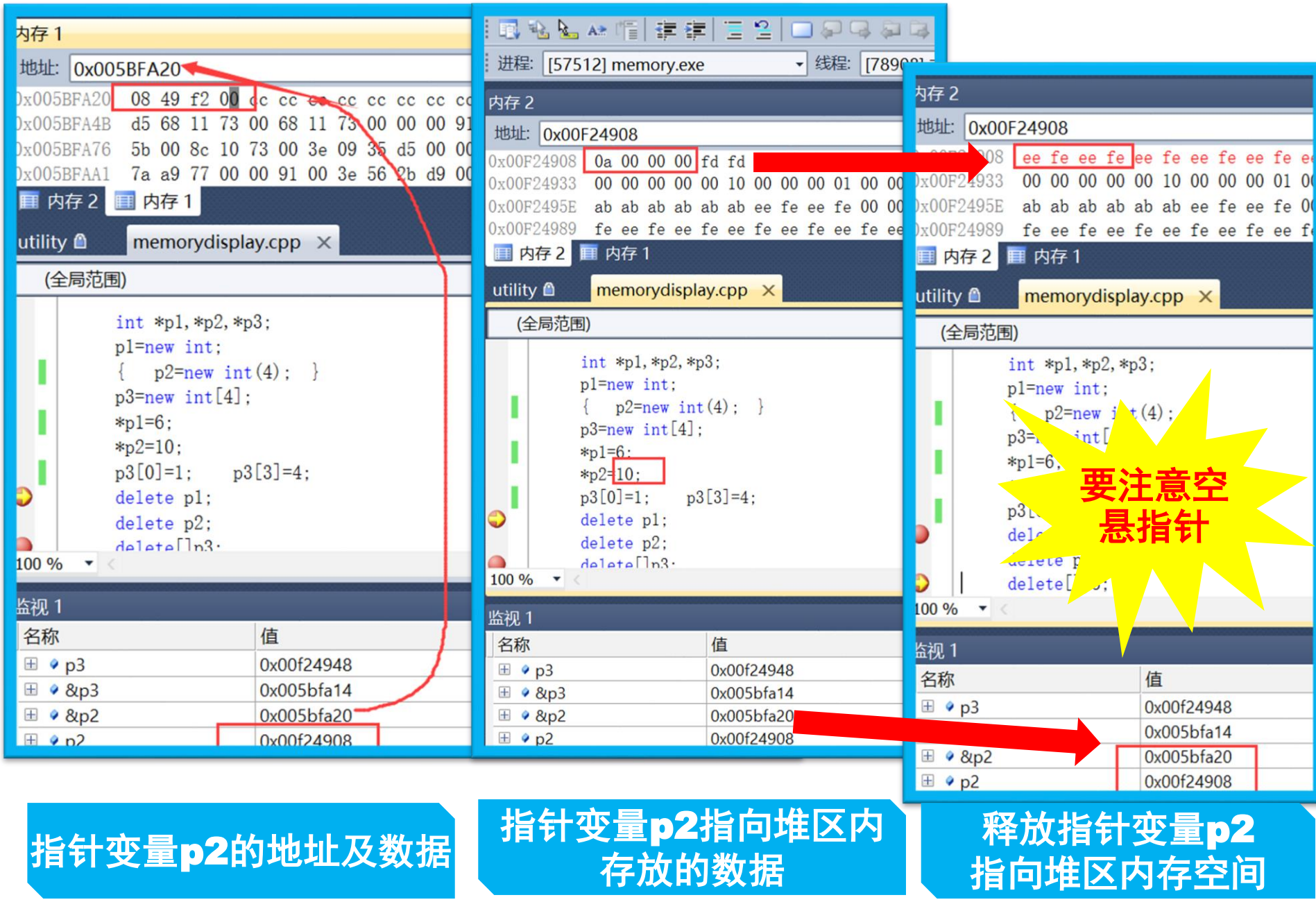
```
delete p1;
delete p2;
delete[ ] p3;
```

只要用new申请的空间，就必须用delete回收

只有用new开辟的空间，才能用delete收回

一次delete即可，避免重复释放

1.4、自由存储区内存管理-示例



指针变量p2的地址及数据

指针变量p2指向堆区内存放的数据

释放指针变量p2指向堆区内存空间

1.3、数组的动态分配与撤销

一维数组的动态分配与撤销

分配：指针变量名=new 类型名[下标表达式];

撤销：delete[ ] 指向该数组的指针变量名;

- 两式中方括号须配对使用。避免内存泄漏和重复释放。
- 若delete语句中少了[ ]，会产生回收不彻底的问题(只回收了首元素所占空间)。内存泄漏！！！
- delete [ ]的方括号中不需要填数组元素数，系统自知。即使写了，编译器也忽略。
- 请注意“下标表达式”不是常量表达式，即它的值不必在编译时确定，可以在运行时确定。

1.3、数组的动态分配与撤销

- 若内存不足，无法正常分配空间，则返回空指针NULL。

```
pc=new char[n];
if (pc!=NULL)
{
    strcpy(pc,“自由存储区内存的动态分配");
    delete [] pc;
}
```

- new还可以开辟自定义数据类型的数据空间：

```
Student *Student_p=new Student;
```

1.3、数组的动态分配与撤销

说一说

```
float *pf= new float(20);
```

开辟存放float型数据的空间，返回该空间地址存放在指针pd中，并且指定该float型数据的初值为20

```
delete pf;
```

撤销以上开辟的float型数据空间

```
char *pc= new char;
```

开辟存放char型数据的空间，返回指向该空间地址的指针

```
delete pc;
```

撤销以上开辟的char型数据空间

```
int *array_p= new int[35];
```

开辟存放一维数组（35个元素）整型数据的空间，指针array中存放数组的首地址

```
delete [ ] array_p;
```

撤销以上开辟的数组空间

1.3、数组的动态分配与撤销

```
#include <iostream>
#include <cstring>
using namespace std;
int main()
{
        int n;
        char *pc;
        cout<<“请输入动态一维数组的元素个数"<<endl;
        cin>>n;
        pc=new char[n];
        strcpy(pc,“自由存储区内存的动态分配");
        cout<<pc<<endl;
        delete [ ] pc;
        return 0;
}
```

动态分配内存，n在运行时由输入值确定

释放pc所指向的n个字符的内存空间

二维数组的动态分配与撤销

```
void display(double **);
void de_allocate(double **);
int m=1,n=1;
int main()
{
        int i,j;
        cout<<"请输入二维数组的行数：";
        cin>>m;
        cout<<endl;
        cout<<"请输入二维数组的列数：";
        cin>>n;
        cout<<endl;
```

1.3、数组的动态分配与撤销

二维数组的动态分配与撤销

二级指针

建立指向二维数组各行的指针数组

```
double **matrix;
matrix = new double*[m];
if ((matrix)==NULL)
{
        cout<<"Could not allocate. Bye ...";
        return -1;
}
for(j=0;j<m;j++)
{
        matrix[j] = new double[n];
        if (matrix[j] == NULL)
        {
                cout << "Could not allocate. Bye ...";
                return -1;
        }
}
```

建立二维数组各行

二维数组的动态分配与撤销

```
    for (i=0;i<m;i++)
            for (j=0;j<n;j++)
                    matrix[i][j]=i*n+j;
    display(matrix);
    de_allocate(matrix);
    return 0;
}
```

撤销

1.3、数组的动态分配与撤销

二维数组的动态分配与撤销

```
void display(double **matrix){
        int i,j;
        for (i=0;i<m;i++)
        {
                for (j=0;j<n;j++)
                cout<<matrix[i][j]<<"\t";
                cout << endl;
        }
}
```

```
void de_allocate(double **matrix){
        int i;
        for (i=0;i<m;i++)       delete[] matrix[i];
        delete[] matrix;
}
```

注意撤销次序，与设置相反

与工程学院 文博 2026